

เจาะลึกสถาปัตยกรรม Claude Code

ถอดรหัสการออกแบบและวิศวกรรมเบื้องหลังระบบ AI Agent ระดับ Production

Safety & Permissions

Core & Agent Loop

Extensibility

API gateway

Access Points

State & Memory

Surface Interfaces

User Terminal

[Read Time: 8 min] | Target Architecture: v2.1.88

จากแค่ช่วยพิมพ์ สู่ตัวแทนที่ลงมือทำ

IDE Assistants (เช่น Copilot)



แนะนำโค้ดทีละบรรทัด
มนุษย์เป็นผู้ขับเคลื่อนหลัก

Agentic Systems (Claude Code)

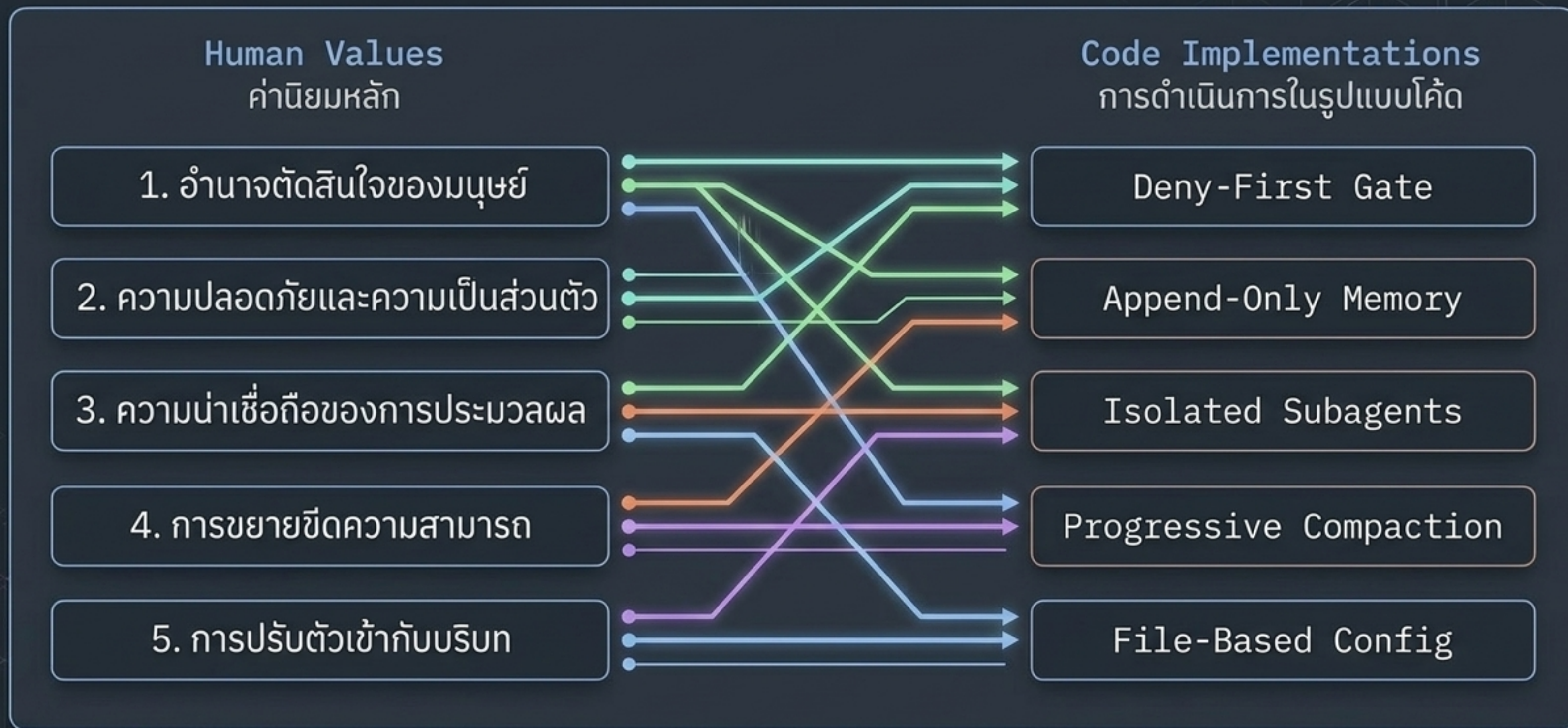


วางแผนแบบหลายขั้นตอน รัน Shell
commands และประเมินผลลัพธ์ด้วยตัวเอง

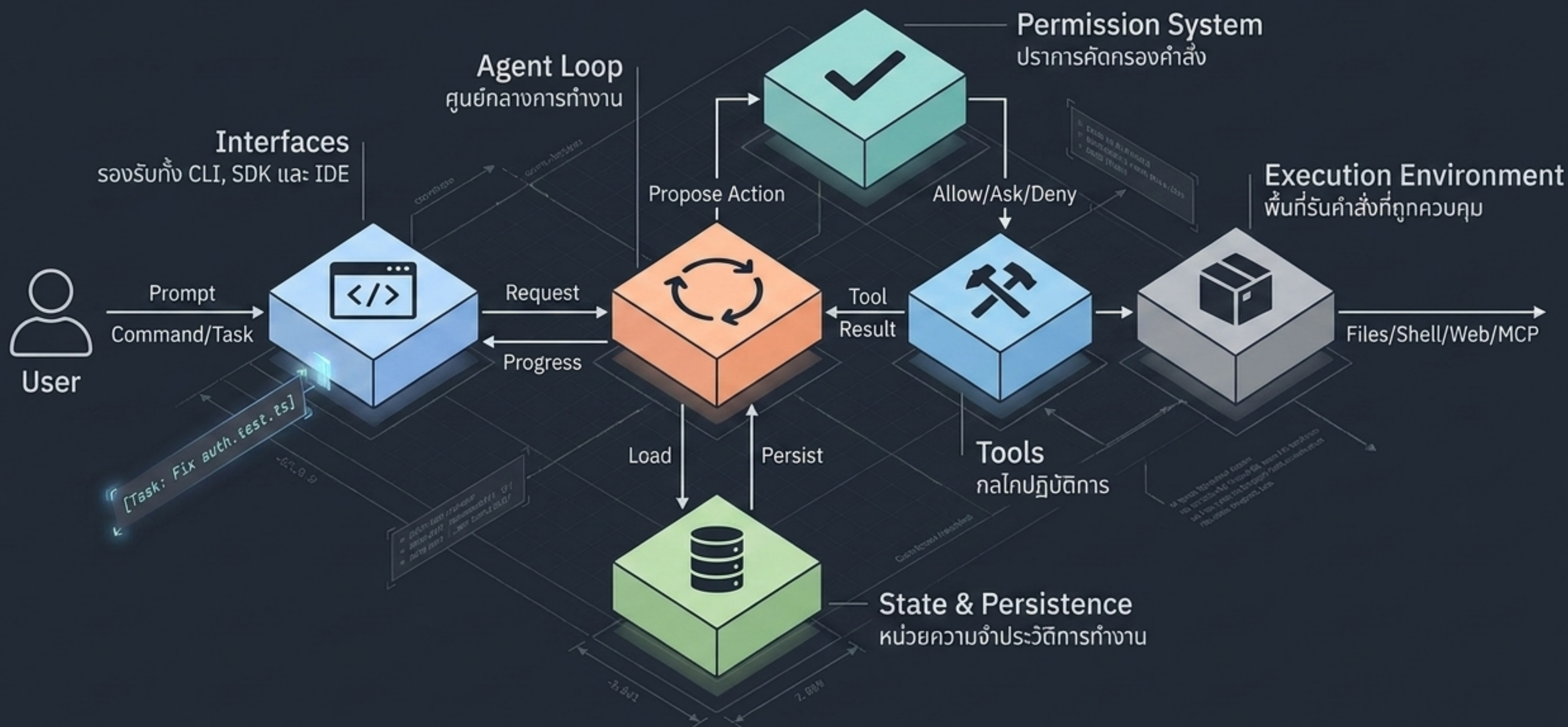
27%

ของงานที่สำเร็จผ่าน Claude Code
คืองานที่นักพัฒนายังจะไม่ยอมเริ่มทำเลยหากไม่มีเครื่องมือนี้

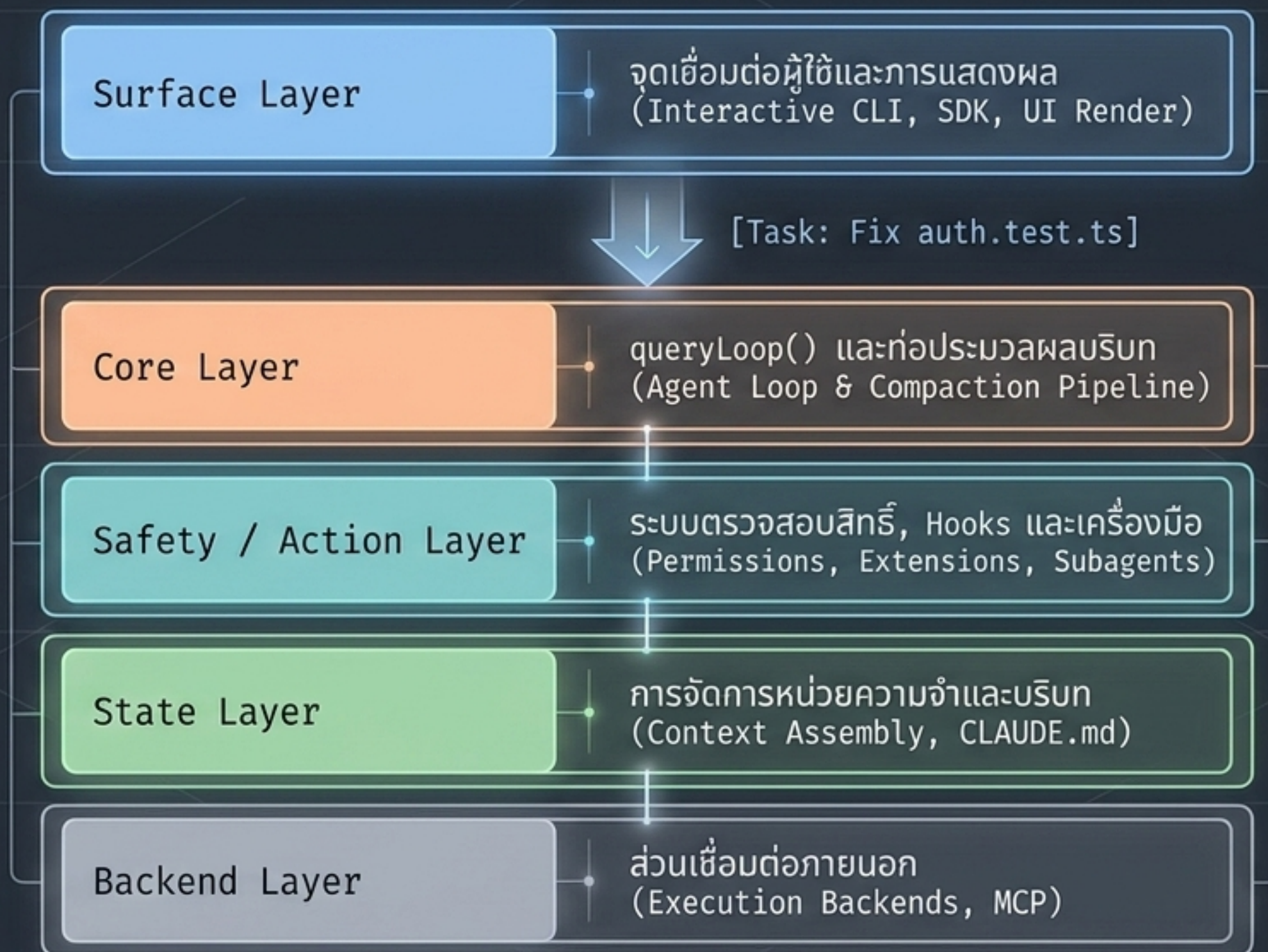
5 ค่านิยมหลักที่กำหนดทิศทางระบบ



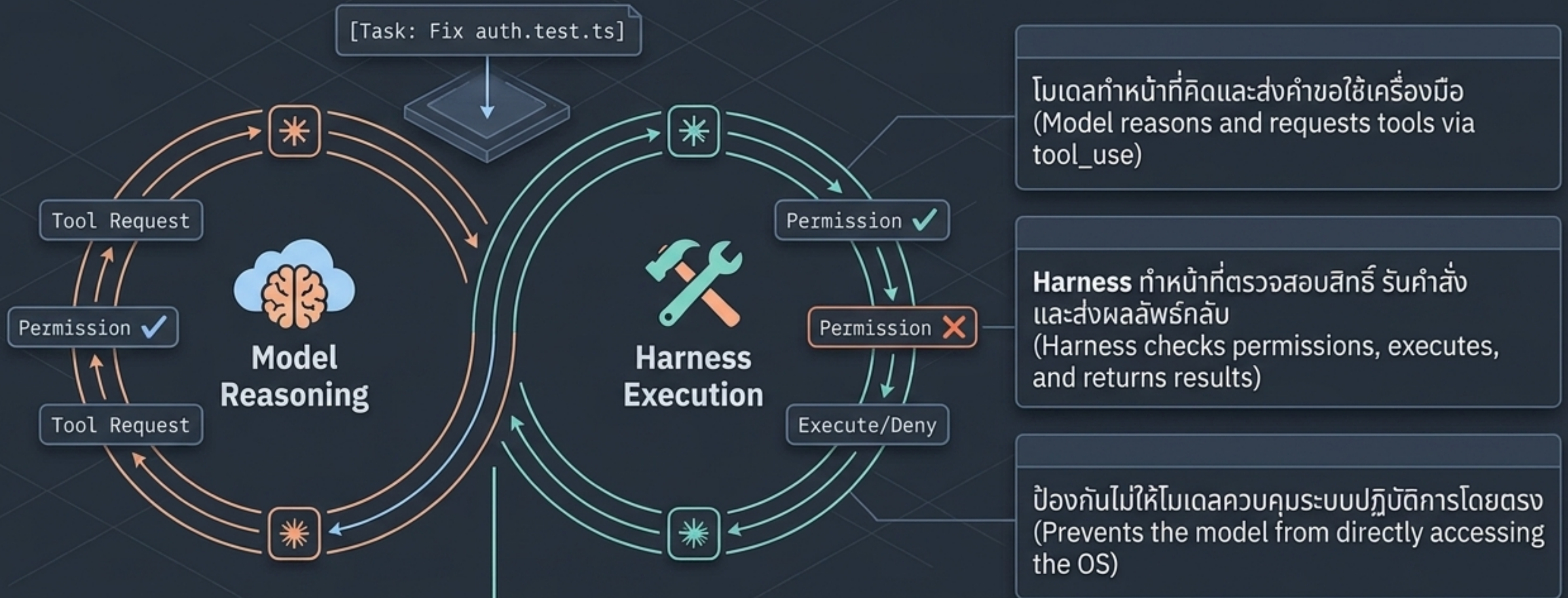
พิมพ์เขียว 7 องค์ประกอบ



สถาปัตยกรรมระบบ 5 ชั้น

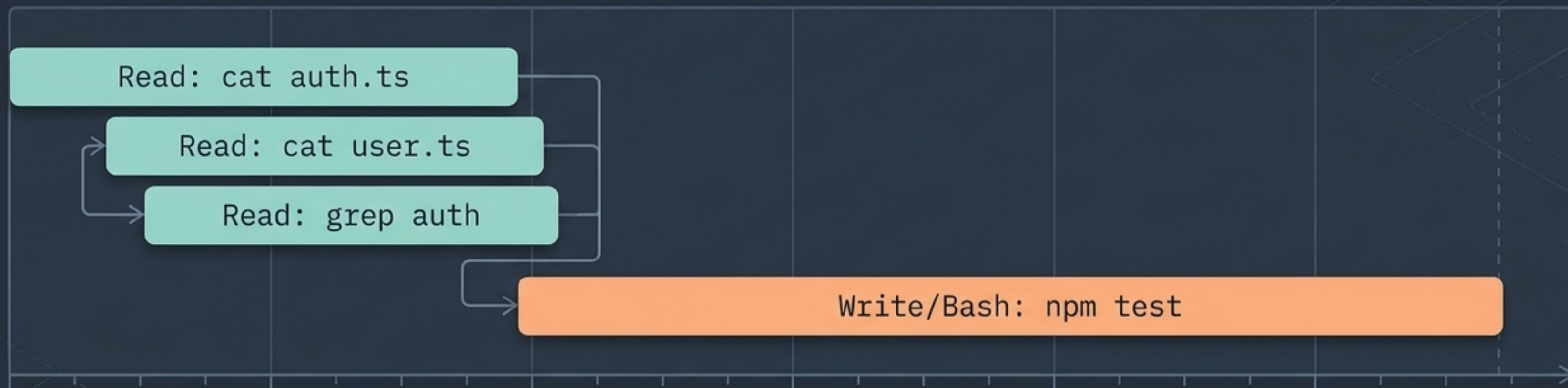


วงจรการทำงานหลัก: ReAct Pattern



ปรัชญาการออกแบบ: แยกตรรกะการตัดสินใจ (AI) ออกจากการลงมือทำ (Harness) อย่างเด็ดขาด

การประมวลผลเครื่องมือ: อ่านพร้อมกัน, เขียนทีละลำดับ



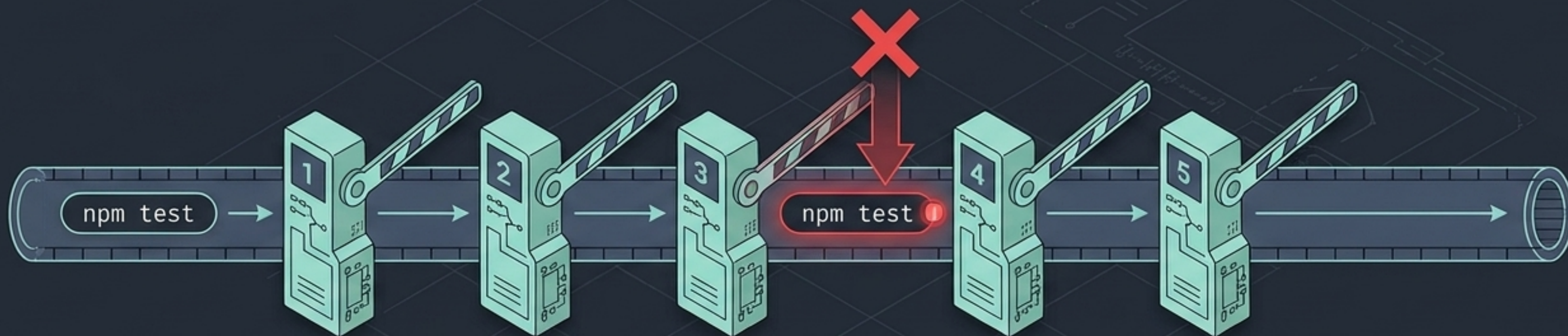
เครื่องมือแบบอ่านอย่างเดียว
(เช่น การดูไฟล์) ทำงานแบบ
ขนานเพื่อลด Latency

คำสั่งที่แก้ไขสถานะระบบ (เช่น
Shell commands) จะถูก
บังคับให้ทำงานตามลำดับเสมอ

⚠ ใช้กลไก Sibling Abort
หากมีคำสั่ง Bash ล้มเหลว จะหยุด
คำสั่งที่กำลังทำงานอยู่ทั้งหมดทันที

ปราการป้องกัน 5 ชั้น: กฎ "ปฏิเสธไว้ก่อน"

เนื่องจากผู้ใช้มีแนวโน้มกดยอมรับคำเตือนถึง 93% ระบบจึงต้องปกป้องตัวเองโดยไม่อาศัยเพียงความระมัดระวังของมนุษย์



1.

1. Pre-filtering

1. Pre-filtering:

ซ่อนเครื่องมือที่ถูกแบน
ก่อนโมเดลจะเห็น

2.

2. Deny-first Rules

2. Deny-first Rules:

กฎปฏิเสธมีอำนาจ
เหนือกฎอนุญาตเสมอ

3.

3. Auto-Mode Classifi

3. Auto-Mode

Classifier: ML ประ
เมินความเสี่ยงของคำสั่ง

4.

4. Hook Pipeline

4. Hook Pipeline:

ตรวจจับและสกัดกั้น
ผ่าน Lifecycle

5.

5. Shell Sandbox

5. Shell Sandbox:

กักบริเวณการรับคำสั่ง

ตารางเปรียบเทียบกลไกส่วนขยาย

กลไก (Mechanism)	จุดเด่น (Unique Capability)	จุดเชื่อมต่อ (Insertion Point)	ต้นทุนบริบท (Context Cost)
Hooks	แทรกแซง Lifecycle	execute()	Zero
Skills	เพิ่มคู่มือเฉพาะทาง	assemble()	Low
Plugins	แพ็คเกจและแจกจ่าย	All	Medium
MCP Servers	เชื่อมต่อบริการภายนอก	model()	High

Context Cost represents the token footprint consumed within the bounded context window.

ลำดับชั้นของหน่วยความจำ: ไปรุ่งใสและแก้ไขได้



ใช้ไฟล์ Markdown แทน Vector Database เพื่อให้ผู้ใช้ตรวจสอบแก้ไข และนำเข้า Version Control ได้ 100%



การบีบอัดข้อมูล 5 ขั้นตอน

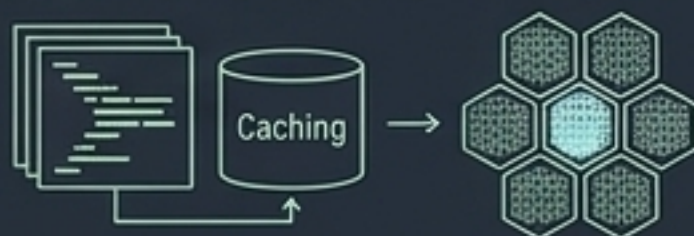
Pastel Orange (Core/Agent)

1. Budget Reduction: ตัดทอนผลลัพธ์เครื่องมือที่ยาวเกินไป



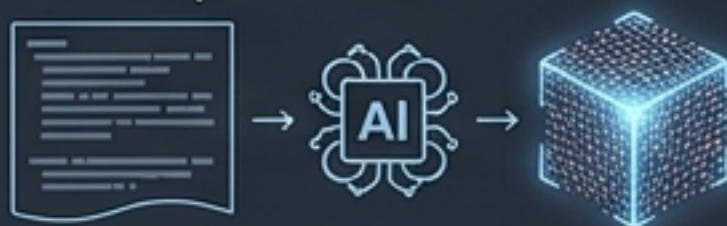
Pastel/ Green (State/Memory)

3. Microcompact: บีบอัดข้อมูลโดยอิงจากระบบ Cache



5. Auto-compact: (Surface Interfaces)

ให้โมเดลสรุปเนื้อหาทั้งหมด



Massive Raw Conversation Data

Pastel Teal (Safety/Permissions)

2. Snip: ตัดประวัติการสนทนาเก่าที่หมดความสำคัญ



4. Context Collapse: (Extensibility)

ซ่อนรายละเอียดเหลือเพียงโครงร่าง



Summary Packet

การกระจายงานและแยกบริบท

Main Conversation

AgentTool

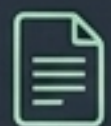
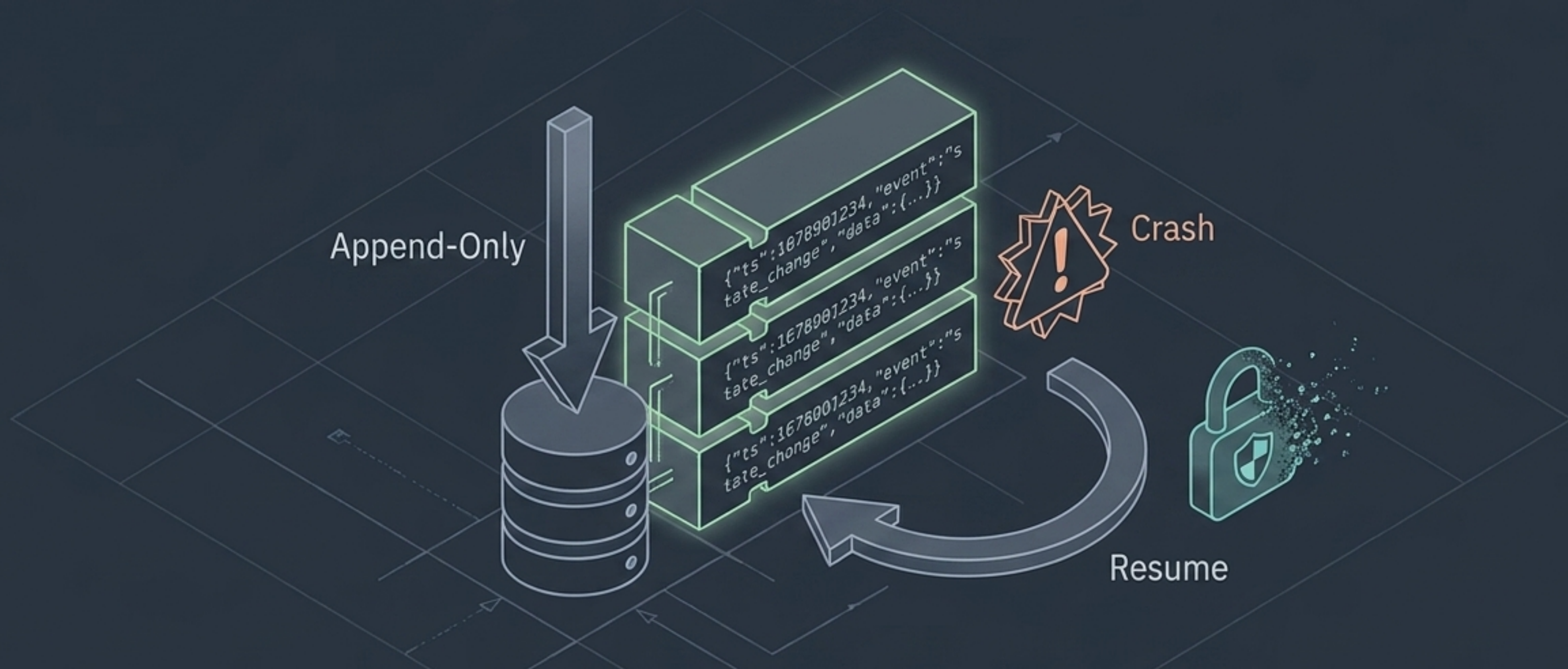
Summary Report Only

Worktree Isolation Sandbox



- สร้าง Worktree จำลองเพื่อให้ Subagent แก้ไขไฟล์โดยไม่กระทบโค้ดหลัก
- ประวัติการสนทนาของ Subagent ถูกเก็บแยกย่อยในไฟล์ .json ของตัวเอง
- ส่งคืนกลับมาแค่ "บทสรุป" เท่านั้น ป้องกันไม่ให้ Context หลักล้น

การบันทึกสถานะของระบบแบบต่อท้าย



บันทึกประวัติเป็นไฟล์ JSONL แบบเพิ่มข้อมูลต่อท้ายเสมอ (Append-only) เพื่อตรวจสอบย้อนหลังได้ 100%



ความปลอดภัยเมื่อกู้คืน: เมื่อ Resume เซสชัน สิทธิ์อนุญาตต่าง ๆ (Permissions) จะไม่ถูกดึงกลับมาด้วย เพื่อบังคับให้ประเมินความปลอดภัยใหม่ทั้งหมด

บทสรุป: บริบทคือคอขวดที่สำคัญที่สุด



ทุกกลไกวิศวกรรมที่ซับซ้อน ล้วนถูกออกแบบมาเพื่อปกป้องทรัพยากรที่มีค่าที่สุด: Context Window

ทิศทางในอนาคตของ AI Agents

Architectural Dimension	Claude Code	Industry Alternatives
Safety/Permissions Isolation Strategy	Layered Policy & Worktrees	Docker Containers (e.g., SWE-Agent)
Core/Agent Loop Execution Engine	Reactive Loop (ReAct)	Explicit State Graphs (e.g., LangGraph)
State/Memory Memory Architecture	File-based Markdown	Vector Embeddings

คำถามที่ยังเปิดกว้าง: สถาปัตยกรรมเหล่านี้ช่วยเพิ่มศักยภาพระยะสั้นได้อย่างมหาศาล แต่เราจะออกแบบระบบอย่างไรเพื่อไม่ให้ทักษะและความเข้าใจระยะยาวของมนุษย์ถดถอยลง?